

Organizational Understanding

No more versions. One continuously evolving cognitive cycle. The unit of value is explanations, not memories.

THE CONSTITUTIONAL SENTENCE

"Maestro exists to become the most accurate, continuously improving explanation of how an organization works. Every signal deepens understanding. Every decision tests understanding. Every outcome revises understanding. Every revision improves future judgment."

This replaces all previous constitutional laws. Not memory. Not adaptation. Not invisible intelligence. Understanding — expressed as explanations. Memories tell you what happened. Explanations tell you why it happened, what changed because of it, and what should happen next. That is what executives buy.

The Permanent Cognitive Cycle

No more V8, V9, V10. One loop. Everything built belongs somewhere inside it. Every future feature, engine, or capability maps to one of these 9 stages. If it does not fit, it does not belong in Maestro.

Stage	What Maestro Does	Existing Engines (V3-V6)
1. Observe	Ingests signals from GitHub, Jira, Slack, Gmail, Signal, etc.	signal_engine.py, historical_engine.py
2. Understand	Builds causal models, detects patterns, infers norms, shapes identity	causal_model.py, consciousness.py, identity.py, wisdom.py
3. Question	Identifies gaps, anomalies, blind spots. Asks the organization directly	organization_direct.py, skepticism.py
4. Predict	Makes predictions about organizational outcomes	prediction_lifecycle.py, prediction_market.py, simulation.py
5. Explain	Synthesizes explanations: why something happened, what it means, what to do	explanation.py, whatative.py (TO BE UPGRADED)
6. Prepare	Drafts execution plans, briefing memos, intervention steps	execution_steps_function.py, preparation.py, adaptive_nudge.py
7. Revise	Updates understanding when predictions miss. Blurs lines, compartmentalizes	blurring_compartmentalizing.py (TO BE UPGRADED)
8. Teach	Shares understanding with the organization via chat, journals, etc.	chat_engine.js, cognition.js, autobiography.js
9. Repeat	The loop never stops. Understanding compounds and replaces old loops	compound_replace.py (TO BE HOOKED INTO live_ingest)

19 cognitive engines already exist across these 9 stages. V3-V6 built them. The Permanent Constitution does not add more engines — it upgrades the EXISTING ones to produce explanations, not just outputs. The loop is the architecture. The engines are the implementation. The explanation is the product.

Step 0 — Fix 2 V6 Wiring Gaps (MUST DO FIRST, 1.5 hours)

2 GAPS FROM ROUND 24 — FIX BEFORE ANYTHING ELSE

Gap 1: Background loop NOT hooked into live_ingest() (30 min). Fix: add `BackgroundLoop.check()` call at end of `oem_state.py live_ingest()`.

Gap 2: DNA NOT referenced in wisdom.py (1 hour). Fix: modify `wisdom.py` to import `organizational_dna.py`, compute `alignment_score` per recommendation.

The 9 Upgrades (Upgrading Existing Engines to Produce Explanations)

The Permanent Constitution does not add engines. It upgrades existing ones so that every output is an explanation, not just a number, a pattern, or a prediction. 9 upgrades. ~15 days.

#	Upgrade	Stage	Effort	What Changes
1	Organizational Explanations	5. Explain	3 days	sowhat.py + narrative.py upgraded to synthesize WHY, not just WHAT. Every confidence score g
2	Four-Level Unknowns	3. Question	1.5 days	curiosity.py upgraded: Known / Known Unknown / Unknown Unknown / Emerging Unknown. R
3	Conversational Curiosity	3. Question	2 days	curiosity.py upgraded: multi-turn conversation, not Q&A. Maestro asks follow-up questions basec
4	Bayesian Institutional Confidence	2. Understand	1.5 days	institutional_confidence.py (V8 #2, to build): living Bayesian confidence with evidence trend, unc
5	Causality Timeline	5. Explain	2 days	memory_timeline.py (V8 #1, to build): every moment answers "why did this permanently change
6	Understanding Topology	8. Teach	2 days	understanding_map.py (V8 #5, to build): network graph, not bars. Connections pulse as Maestro l
7	Organizational Blind Spots	3. Question	1 day	NEW. "Engineering has raised deployment risk 11 times. Leadership has ignored it 11 times. Blin
8	Institutional Gravity	2. Understand	2 days	NEW. Visualize centers of gravity (people, projects, decisions, knowledge, approvals, customers).
9	Memory Compression (Deep)	2. Understand	2 days	memory_compression.py upgraded: 10M events -> 200 principles -> 43 reflexes -> 19 institutiona
TOTAL 9 upgrades		5 stages	~17 days	From 19 engines producing outputs to 19 engines producing explanations

#1 Organizational Explanations — the biggest moat

Principle

The constitutional sentence: "Every signal deepens understanding." Understanding is not a score. It is an EXPLANATION. "Why are engineering estimates always wrong?" should produce: "We've observed this for eleven months. Here is why: PRs enter review queue -> cross-team dependency -> architecture ownership -> late QA -> missed estimate." That explanation is synthesized from hundreds of observations. Nobody stores explanations. Everyone stores documents. This is a gigantic difference.

Codebase gap

sowhat.py (197 lines) produces "consequence_if_ignored" — a single sentence. narrative.py (332 lines) produces daily highlights — a list of events. Neither produces CAUSAL EXPLANATIONS — multi-step reasoning chains that explain WHY something happens. The gap: Maestro can say "bottlenecks correlate with velocity drops" but cannot explain "WHY does this bottleneck cause velocity drops in THIS organization — what is the causal chain from PR to review queue to dependency to missed estimate?"

Build

CREATE `backend/maestro_oem/explanations.py` — the ExplanationEngine. Takes a question ("Why are engineering estimates always wrong?") and synthesizes a causal explanation: (1) retrieves relevant causal chains (from `causal.py`), (2) retrieves signal patterns (from `pattern.py`), (3) retrieves evidence (from `evidence_graph.py`), (4) composes a multi-step explanation graph: PR -> review queue -> cross-team dependency -> architecture ownership -> late QA -> missed estimate. Each step has `evidence_count` + `confidence`. CREATE GET `/api/oem/explain?q=...` endpoint. MODIFY `static/js/ask_v2.js` — "why" questions route to the Explanation engine and render the explanation as a visual chain (not a text paragraph). MODIFY all confidence displays (TODAY, Cognition, LEARN) — every confidence score gets a "Why?" link that opens the explanation.

Acceptance test

1) GET `/api/oem/explain?q=Why+are+engineering+estimates+always+wrong` returns a multi-step causal explanation (3+ steps, each with `evidence_count` + `confidence`). 2) Each step references real model data (not hardcoded). 3) ASK v2 renders the explanation as a visual chain. 4) Every confidence score in the UI has a "Why?" link that opens the explanation. 5) V5 litmus: no new panel — enhances ASK v2 + existing confidence displays. 6) Permanent Constitution litmus: does this make the customer say "Maestro understands our company"? YES — explanations are the proof of understanding.

Effort

3 days (2 days explanation-synthesis engine + 0.5 day API + 0.5 day frontend visual chain)

Dependencies

`causal.py` (for causal chains). `evidence_graph.py` (for evidence). `pattern.py` (for patterns). `sowhat.py` (for consequence logic). `narrative.py` (for narrative composition).

#2 Four-Level Unknowns — Known / Known Unknown / Unknown Unknown / Emerging Unknown

Principle

Currently Maestro reports "We don't understand HR." That is only level one. Four levels: (1) Known — "Deployment pipeline." (2) Known Unknown — "Executive hiring." (3) Unknown Unknown — "There are probably hidden decision mechanisms we have never observed." (4) Emerging Unknown — "A completely new organizational behavior appeared yesterday. We don't know what it is yet." That is much richer scientifically.

Codebase gap

`curiosity.py` (184 lines) identifies untested assumptions and unmeasured domains. But it has one level: "we don't understand X." The gap: no four-level taxonomy. No "emerging unknown" detection (a new signal pattern that appeared yesterday and Maestro cannot classify). No "unknown unknown" estimation (what decision mechanisms might exist that Maestro has never observed?).

Build

UPGRADE `backend/maestro_oem/curiosity.py` — add a `classify_unknowns()` method that categorizes organizational areas into 4 levels: Known (coverage > 60%), Known Unknown (coverage 10-60%, Maestro knows it doesn't understand), Unknown Unknown (coverage < 10%, Maestro suspects hidden mechanisms), Emerging Unknown (new signal

pattern in last 7 days that doesn't match any existing learning object). CREATE GET /api/oem/unknowns?levels=all endpoint (returns all 4 levels). MODIFY static/js/today.js — "What Maestro doesn't know yet" section now shows 4 levels with different visual treatment.

Acceptance test

1) GET /api/oem/unknowns?levels=all returns 4 arrays: known, known_unknowns, unknown_unknowns, emerging_unknowns. 2) Each item has area + coverage + reason. 3) Emerging unknowns have "detected_at" (last 7 days). 4) TODAY shows 4 levels with different treatment. 5) V5 litmus: no new panel. 6) Constitution litmus: does this make the customer trust Maestro MORE? YES — 4-level honesty is more scientifically rigorous than 1-level.

Effort

1.5 days (1 day four-level engine + 0.5 day API + frontend)

Dependencies

curiosity.py (upgrade). institutional_confidence.py (V8 #2, for coverage thresholds). signal history (for emerging unknown detection).

#3 Conversational Curiosity — multi-turn, not Q&A;

Principle

Instead of Question -> Answer, build Question -> Discussion -> Clarification -> Understanding Updated. "Why were approvals doubled?" "Holiday backlog." "Was this planned?" "No." "Did the backlog change delivery quality?" "Slightly." "Thank you. Institutional understanding updated." That feels alive.

Codebase gap

curiosity.py generates single questions with text input. The answer becomes a signal. But there is no FOLLOW-UP — Maestro does not ask clarifying questions based on the answer. The gap: no conversational state. No multi-turn curiosity. Maestro asks once and moves on.

Build

UPGRADE backend/maestro_oem/curiosity.py — add follow_up() method that takes the previous question + answer and generates a clarifying follow-up. CREATE POST /api/oem/curiosity/follow-up endpoint (accepts question_id + answer, returns follow_up_question or "understanding_updated: true"). MODIFY static/js/today.js — "Maestro has questions" card becomes a conversation: question -> input -> follow-up -> input -> "Thank you. Understanding updated." (max 3 turns per topic).

Acceptance test

1) POST /api/oem/curiosity/follow-up returns either a follow_up_question or {"understanding_updated": true, "signal_created": true}. 2) Follow-up questions reference the previous answer (not generic). 3) Max 3 turns per topic (Maestro does not interrogate). 4) After the conversation, understanding is updated (a new signal is created AND the relevant model component is adjusted). 5) TODAY shows a conversation flow (not a single Q&A). 6) Constitution litmus: does this make the customer say "Maestro knows how our company works"? YES — the org teaches Maestro through conversation.

Effort

2 days (1.5 days conversational engine + 0.5 day API + frontend)

Dependencies

curiosity.py (upgrade). signal.py (for creating signals from answers).

#4 Bayesian Institutional Confidence — living, not static

Principle

Not "91%." Instead: "Understanding 91%. Evidence: Growing. Uncertainty: Low. Last revised: Yesterday. Confidence trend: up. Prediction calibration: 93%. Competing explanation: 8%." Now confidence is living, not static.

Codebase gap

confidence.py (472 lines) uses Beta-Binomial posterior for per-LAW confidence. But it is a single number, not a living state. The gap: no evidence trend (growing/stable/declining), no uncertainty band, no competing explanation percentage, no last-revised timestamp, no prediction calibration per domain.

Build

CREATE backend/maestro_oem/institutional_confidence.py (V8 #2, to build) with BAYESIAN per-domain confidence: {understanding (0-1), evidence_trend (growing/stable/declining), uncertainty (low/medium/high), last_revised (timestamp), confidence_trend (up/down/flat), prediction_calibration (0-1), competing_explanation_pct (0-1), narrative}. CREATE GET /api/oem/confidence endpoint. MODIFY static/js/today.js — "Understanding" section shows living confidence with trend indicators (not static bars).

Acceptance test

1) GET /api/oem/confidence returns per-domain confidence with ALL 8 fields. 2) evidence_trend computed from signal rate over 30 days. 3) competing_explanation_pct computed from competing hypotheses (when V7.1 is built — until then, honest "insufficient competing hypotheses"). 4) TODAY shows living confidence with trend indicators. 5) Constitution litmus: does this make the customer trust Maestro? YES — living Bayesian confidence is more honest than a static number.

Effort

1.5 days (1 day Bayesian engine + 0.5 day API + frontend)

Dependencies

confidence.py (for Beta-Binomial). prediction_lifecycle.py (for calibration). signal history (for evidence trend).

#5 Causality Timeline — every moment answers "why did this permanently change us?"

Principle

Today: Event -> Learning. Eventually: Observation -> Decision -> Prediction -> Outcome -> Institution Changed -> Future Decisions Changed. Every card should answer "Why did this permanently change us?" That becomes true institutional memory.

Codebase gap

memory_timeline.py (V8 #1, to build) produces moment cards with date + event + prediction + reality + learning. But the "institution_change" field is a string, not a causal chain. The gap: no link between the learning and the FUTURE decisions that changed because of it. "Every deployment recommendation since then uses this rule" is asserted but not traced.

Build

UPGRADE backend/maestro_oem/memory_timeline.py (V8 #1, to build) — each moment card gets a causal_chain field: {observation, decision, prediction, outcome, institution_change, future_decisions_affected (list of recommendation IDs that changed)}. CREATE GET /api/oem/memory-timeline endpoint. MODIFY the "What we've learned" surface — each moment card shows the full causal chain as a visual flow.

Acceptance test

1) GET /api/oem/memory-timeline returns moments with causal_chain field (6 steps). 2) future_decisions_affected references real recommendation IDs. 3) Each moment card shows the causal chain visually. 4) Constitution litmus: does this make the customer say "Maestro knows how our company works"? YES — it shows WHY the organization changed, not just THAT it changed.

Effort

2 days (1.5 days causal-chain tracing + 0.5 day frontend)

Dependencies

memory_timeline.py (V8 #1). prediction_lifecycle.py (for predictions). adaptive_nudge.py (for interventions).

#6 Understanding Topology — network graph, not bars

Principle

Instead of bars, imagine a living network. Engineering -> Platform -> Security -> Legal -> Sales. Connections pulse as Maestro learns. Users literally watch understanding spread. Almost like watching neurons grow. Nobody has that today.

Codebase gap

`understanding_map.py` (V8 #5, to build) produces horizontal bars per domain. But bars do not show RELATIONSHIPS between domains. The gap: no network visualization showing how understanding of one domain connects to another. No "pulse" when a new signal deepens understanding.

Build

UPGRADE `backend/maestro_oem/understanding_map.py` (V8 #5) — add `topology()` method that returns nodes (domains with coverage) + edges (cross-domain dependencies from `evidence_graph.py`). Each edge has a strength (how many cross-domain signals exist). CREATE GET `/api/oem/understanding-topology` endpoint. CREATE `static/js/understanding_topology.js` — renders a force-directed network graph (domains as nodes, cross-domain signals as edges). Nodes are sized by coverage. Edges pulse when new cross-domain signals arrive. MODIFY `static/js/learn.js` — Understanding Map section becomes Understanding Topology (network, not bars).

Acceptance test

1) GET `/api/oem/understanding-topology` returns nodes + edges with strength. 2) LEARN surface shows a network graph (not bars). 3) Nodes are sized by coverage. 4) Edges have strength from cross-domain signal count. 5) Constitution litmus: does this make the customer say "Maestro knows how our company works"? YES — they literally watch understanding spread.

Effort

2 days (1 day topology engine + 0.5 day API + 0.5 day frontend network visualization)

Dependencies

`understanding_map.py` (V8 #5). `evidence_graph.py` (for cross-domain edges).

#7 Organizational Blind Spots — evidence exists, org ignores it

Principle

Different from Unknown Unknowns. Unknown Unknown: "No evidence." Blind Spot: "Evidence exists. Organization ignores it." Example: "Engineering has raised deployment risk 11 times. Leadership has ignored it 11 times. Blind spot detected." This is one of the most valuable insights a CEO could ever receive.

Codebase gap

No module tracks ignored evidence. contradiction.py detects contradictions between beliefs and behavior. But it does not track PATTERNS OF IGNORED EVIDENCE — signals that were raised (in Slack, in Jira, in postmortems) but never resulted in action. The gap: Maestro cannot say "this risk has been raised 11 times and ignored 11 times."

Build

CREATE backend/maestro_oem/blind_spots.py — the BlindSpotEngine. Scans for signals that indicate a raised risk/concern (Slack messages with risk keywords, Jira issues with "risk" label, postmortem recommendations) and checks whether any action was taken (was a related PR merged? was a policy changed? was a task created?). If raised_count > 3 AND action_count == 0, flag as a blind spot. CREATE GET /api/oem/blind-spots endpoint. MODIFY static/js/today.js — if blind spots exist, show them as a high-priority card: "Engineering has raised deployment risk 11 times. Leadership has not acted. This is a blind spot."

Acceptance test

1) GET /api/oem/blind-spots returns blind_spots array. 2) Each has area, raised_count, action_count, evidence (signal references), severity. 3) raised_count > 3 AND action_count == 0 required for blind spot status. 4) TODAY shows blind spot card when detected. 5) Constitution litmus: does this make the customer say "Maestro knows how our company works"? YES — it sees what the org ignores.

Effort

1 day (0.5 day blind-spot detection + 0.5 day API + frontend)

Dependencies

signal history (for raised risks). pattern.py (for recurrence detection). contradiction.py (for existing contradiction logic).

#8 Institutional Gravity — visualize centers of gravity

Principle

Every organization has centers of gravity. Not org charts. Real gravity. People, projects, decisions, knowledge, approvals, customers — everything bends around certain people. Maestro visualizes it. When gravity shifts, the organization changes. That is measurable.

Codebase gap

consciousness.py (204 lines) tracks organizational state (attention, knowledge, trust, etc.). personality.py (247 lines) infers traits. But neither visualizes GRAVITY — the concentration of organizational activity around specific people. The gap: no module identifies "Alice is a center of gravity for auth decisions" or "gravity shifted from Alice to Bob last month."

Build

CREATE backend/maestro_oem/institutional_gravity.py — the GravityEngine. For each person (from signal actors), compute gravity_score based on: (1) decision_count (how many decisions involve them), (2) knowledge_holding (how many domains they hold knowledge in), (3) approval_count (how many approvals pass through them), (4) cross_team_connections (how many teams they interact with). Track gravity over time — when gravity shifts (one person's score drops > 20% while another's rises), flag it. CREATE GET /api/oem/gravity endpoint. MODIFY static/js/learn.js — add "Centers of gravity" section showing top 5 people by gravity_score + any recent shifts.

Acceptance test

1) GET /api/oem/gravity returns people with gravity_score + trend. 2) At least 3 people with score > 0. 3) Gravity shifts detected (person A drops, person B rises). 4) LEARN shows "Centers of gravity" section. 5) Constitution litmus: does this make the customer say "Maestro knows how our company works"? YES — it sees who the org actually revolves around (not the org chart).

Effort

2 days (1.5 days gravity computation + 0.5 day API + frontend)

Dependencies

signal.py (for actor data). consciousness.py (for state). evidence_graph.py (for knowledge holding). model.py (for approval network).

#9 Memory Compression (Deep) — 10M events to 7 cultural axioms

Principle

10 million events -> 200 principles -> 43 reflexes -> 19 institutional truths -> 7 cultural axioms. Maestro compresses experience exactly like the human brain does. That is an extraordinary product.

Codebase gap

memory_compression.py (152 lines, V4) compresses into truths/habits/mistakes (3 categories). But the compression is shallow — it does not produce PRINCIPLES, REFLEXES, TRUTHS, and AXIOMS at different abstraction levels. The gap: no multi-level compression that mirrors how human institutional knowledge forms (from raw events to cultural axioms).

Build

UPGRADE backend/maestro_oem/memory_compression.py — add 5-level compression: (1) Events (raw signals, filtered for relevance by forgetting.py), (2) Principles (validated laws with ≥ 5 runtimes — from law.py), (3) Reflexes (patterns so strong they are automatic — from pattern.py with strength > 0.8), (4) Institutional Truths (principles that have survived ≥ 1 year without failure — from principles.py when built), (5) Cultural Axioms (truths so fundamental they define the organization — from identity.py beliefs with drift < 0.1). CREATE GET /api/oem/compression-deep endpoint. MODIFY static/js/learn.js — add "How your organization compresses experience" section showing the 5 levels with counts and examples.

Acceptance test

1) GET /api/oem/compression-deep returns 5 levels: events_count, principles_count, reflexes_count, truths_count, axioms_count + examples for each. 2) Each level has a lower count than the one below (compression). 3) LEARN shows the 5 levels. 4) Constitution litmus: does this make the customer say "Maestro knows how our company works"? YES — it shows the org's compressed wisdom at every abstraction level.

Effort

2 days (1.5 days multi-level compression + 0.5 day API + frontend)

Dependencies

memory_compression.py (upgrade). law.py (for principles). pattern.py (for reflexes). identity.py (for axioms). forgetting.py (for event filtering).

Build Order

Step	Upgrade	Effort	Stage	Key Output
0	Fix V6 wiring gaps	1.5 hours	9. Repeat	Background loop in live_ingest + DNA in wisdom.py
1	#1 Organizational Explanations	3 days	5. Explain	WHY, not just WHAT. Every confidence gets a "Why?" link.
2	#2 Four-Level Unknowns	1.5 days	3. Question	Known / Known Unknown / Unknown Unknown / Emerging Unknown
3	#3 Conversational Curiosity	2 days	3. Question	Multi-turn conversation, not Q&A
4	#4 Bayesian Institutional Confidence	1.5 days	2. Understand	Living confidence with trend + uncertainty + calibration
5	#5 Causality Timeline	2 days	5. Explain	Every moment answers "why did this permanently change us?"
6	#6 Understanding Topology	2 days	8. Teach	Network graph. Watch understanding spread.
7	#7 Organizational Blind Spots	1 day	3. Question	"Raised 11 times. Ignored 11 times. Blind spot."
8	#8 Institutional Gravity	2 days	2. Understand	Centers of gravity. When gravity shifts, the org changes.
9	#9 Memory Compression (Deep)	2 days	2. Understand	10M events -> 200 principles -> 43 reflexes -> 19 truths -> 7 axioms
TOTAL	9 upgrades + V6 fixes	~17 days + 1.5 hours	6 stages	From outputs to explanations. From memory to understanding.

The Permanent Constitution Litmus Test

DOES THIS MAKE THE CUSTOMER SAY "MAESTRO UNDERSTANDS OUR COMPANY"?

The single test that replaces all previous litmus tests. Not "is the UI simpler?" Not "does the org improve?" Not "is the understanding more accurate?" All of those are necessary but insufficient. The final test: does the customer experience Maestro as something that UNDERSTANDS their company?

Understanding is demonstrated through EXPLANATIONS. If the customer asks "why?" and Maestro can explain — with evidence, with causal chains, with historical analogues — then Maestro understands. If Maestro can only show a number, a pattern, or a prediction, it does not understand. It observes. Observing is necessary but insufficient. Understanding is the product.

Every upgrade in this constitution must pass this test. If an upgrade produces a number without an explanation, it fails. If it produces a prediction without a "why," it fails. If it produces a recommendation without evidence the customer can inspect, it fails. The bar is understanding, expressed as explanations.

The End

"MAESTRO UNDERSTANDS OUR COMPANY"

One year later, the customer does not say "We use Maestro." They say "Maestro understands our company." That sentence is the product. Not a dashboard. Not a copilot. Not an intelligence layer. An understanding — continuously improving, evidence-backed, explainable, and irreplaceable.

The 19 engines remain internally. The customer sees one thing: explanations. They ask "why?" and Maestro explains — with causal chains, with evidence, with historical analogues, with honest confidence and honest gaps. They see their organization's understanding grow on a topology map. They revisit moments where the institution changed. They answer Maestro's questions and watch understanding update in real time. They see blind spots Maestro detected that they were ignoring. They see centers of gravity shift. They see 10 million events compressed into 7 cultural axioms.

The constitutional sentence: "Maestro exists to become the most accurate, continuously improving explanation of how an organization works. Every signal deepens understanding. Every decision tests understanding. Every outcome revises understanding. Every revision improves future judgment."

No more versions. One loop. Observe, Understand, Question, Predict, Explain, Prepare, Revise, Teach, Repeat. Forever.

Build order: Step 0 (fix V6 gaps, 1.5 hours) -> #1 Explanations (3 days) -> #2 Four-Level Unknowns (1.5 days) -> #3 Conversational Curiosity (2 days) -> #4 Bayesian Confidence (1.5 days) -> #5 Causality Timeline (2 days) -> #6 Understanding Topology (2 days) -> #7 Blind Spots (1 day) -> #8 Institutional Gravity (2 days) -> #9 Memory Compression Deep (2 days). Total ~17 days + 1.5 hours. Then ship the 90-day pilot. The product is Organizational Understanding. Build it.